

71. Une carte plus grande que l'écran : la caméra

Le héros a une place dans le monde ; la caméra le suit ; l'écran montre le monde moins la caméra.

1. Ce qu'il faut comprendre

Dans un vrai jeu, le héros **marche** dans un monde plus large que l'écran, et le décor glisse pour le garder en vue. Il y a donc **deux** positions : celle du héros **dans le monde** (`wx`), de 0 à 248 sur une carte de 256 pixels), et celle de la **caméra**, `cam` : le bord gauche de ce qu'on voit.

Ce qui est nouveau ici : l'écran = le monde - la caméra. Le décor, on le fait glisser de `cam` : `defiler(cam, 0)`. Le héros, on le pose à `wx - cam` sur l'écran. Exemple : héros en `wx = 150`, caméra en `cam = 74` : il est dessiné au pixel $150 - 74 = 76$, le milieu de l'écran.

La caméra **suit** le héros : elle veut le garder au milieu, donc `cam = wx - 76`. Mais elle ne doit pas sortir de la carte. À gauche, si `wx < 76`, `wx - 76` passerait sous zéro (pour `wx = 30`, $30 - 76$ donnerait 210 : la règle de l'octet, chapitre 1) : on la bloque à 0. À droite, l'écran fait 160 pixels et la carte 256 : la caméra ne va pas plus loin que $256 - 160 = 96$.

Déroulons. `wx = 30` : trop à gauche, `cam = 0`, le héros est à 30 sur l'écran — il marche, le décor ne bouge pas. `wx = 150` : `cam = 74`, héros au milieu, le décor défile. `wx = 240` : `wx - 76 = 164`, plus que 96, donc `cam = 96`, et le héros est à $240 - 96 = 144$: il marche de nouveau seul vers le bord.

C'est la caméra de presque tous les jeux de plateforme. Les chiffres du sol disent où l'on est dans le monde.

À toi : au lieu de garder le héros pile au milieu, laisse-le libre entre les pixels 60 et 92 de l'écran (une « zone morte ») : la caméra ne bouge que s'il en sort.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **707 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	64 o	← la console, à l'allumage — la mise en route de la console
main()	217 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), nombre() — trouve l'adresse d'une case de l'écran
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	40 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran

```

LireManette      37 o ← bouton() - lit les huit boutons
Tuiles           192 o ← CopierTuiles - 10 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert  10 o ← InstallerTransfert - la petite routine qui copie les lutins
Lettres gravées : 0123456789
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes
Fonctions du langage appelées :
  sprite() × 1    27 o sur place, dans main()
  poser()  × 2    26 o sur place, dans main() + AdresseCase (10 o)
  nombre() × 1    25 o sur place, dans main() + AdresseCase (10 o), EcrireNombre (33 o)
  ecran()  × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
  bouton() × 2    10 o sur place, dans main() + LireManette (37 o)
  defiler() × 1    7 o sur place, dans main()
  image()  × 1    6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un héros marche dans un monde plus large que l'écran ;
// une « caméra » le suit, sans jamais sortir de la carte.
//
// Ce qui est nouveau : deux positions.
// wx = la place du héros DANS LE MONDE (0 à 248).
// cam = le bord gauche de ce qu'on voit, dans le monde (0 à 96).
// Sur l'écran, le héros est à : wx - cam (le monde moins la caméra).
// Exemple : wx = 150, cam = 74 → dessiné au pixel 150 - 74 = 76, le milieu.

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <ecran>      // éteint ou rallume l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <nombre>     // écrit un nombre en chiffres
#include <bouton>     // lit un bouton de la manette
#include <defiler>    // fait glisser tout le fond
#include <sprite>     // place un lutin de 8 × 8 au pixel près

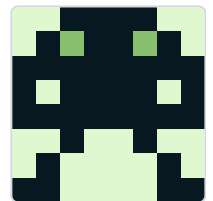
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {
  "..####..",
  ".#-##-#.",
  "#####",
  "#.####.#",
  "#####",
  "..#..#..",
  ".#...#.",
  "##....##",
};

```



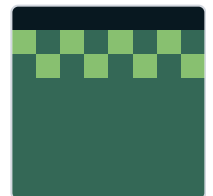
à l'écran

3. UN DESSIN — SOL

```

Tuile SOL = {
  "33333333", "12121212", "21212121", "22222222",
  "22222222", "22222222", "22222222", "22222222",
};

```



à l'écran

4. UNE CONSTANTE — MILIEU

```

const uint8_t MILIEU = 76;          // le pixel du milieu de l'écran, pour le héros

```

5. UNE CONSTANTE — CAM_MAX

```

const uint8_t CAM_MAX = 96;        // 256 - 160 : la caméra ne va pas plus loin

```

6. UNE VARIABLE — WX

```
uint8_t wx = 8;           // le héros, DANS LE MONDE : 0 à 248
```

7. UNE VARIABLE — CAM

```
uint8_t cam = 0;          // le bord gauche de ce qu'on voit, dans le monde : 0 à 96
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    ecran(0);                // Dessin écran éteint.
    for (uint8_t c = 0; c < 32; c++) { // Toute la carte (32 colonnes) :
        poser(c, 12, SOL);      // deux rangées de sol,
        poser(c, 13, SOL);
        if (c % 4 == 0) nombre(c, 10, c / 4, 1); // et un chiffre toutes les 4 colonnes.
    }
    ecran(1);                // On rallume.

    while (true) {           // La boucle du jeu :
        image();

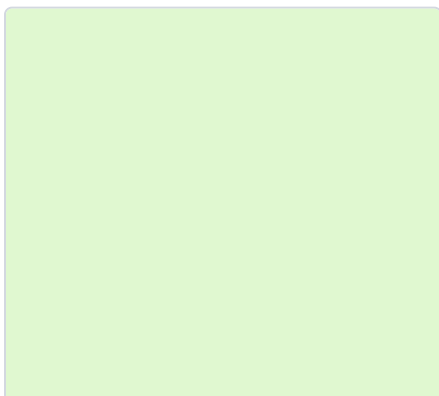
        if (bouton(DROITE) && wx < 248) wx++; // le héros marche dans le monde
        if (bouton(GAUCHE) && wx > 0) wx--;

        // La caméra suit le héros, sans sortir de la carte.
        if (wx < MILIEU) {
            cam = 0;          // trop à gauche : la caméra reste au bord
                               // (wx - 76 passerait sous 0 et donnerait 200 et quelques)
        } else if (wx - MILIEU > CAM_MAX) {
            cam = CAM_MAX;    // trop à droite : elle s'arrête à 96
        } else {
            cam = wx - MILIEU; // sinon : le héros au milieu
        }
        // Déroulé : wx = 30 → cam = 0, héros à 30 ; wx = 150 → cam = 74, héros à 76 ;
        //           wx = 240 → 164 > 96 → cam = 96, héros à 144.

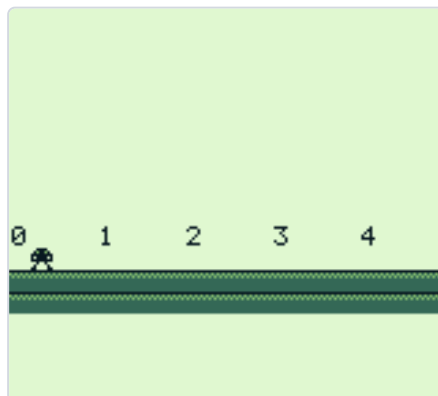
        defiler(cam, 0);      // le décor : décalé de cam
        sprite(0, wx - cam, 88, HEROS); // le héros : le monde moins la caméra
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



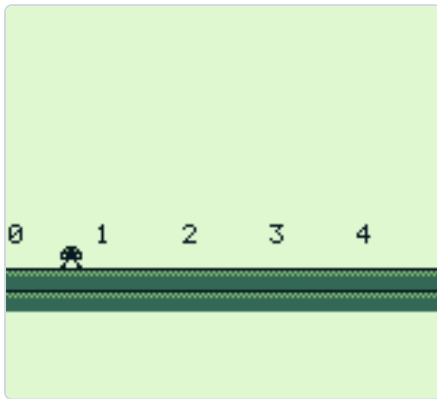
la toute première image — la console vient de démarrer — image 1



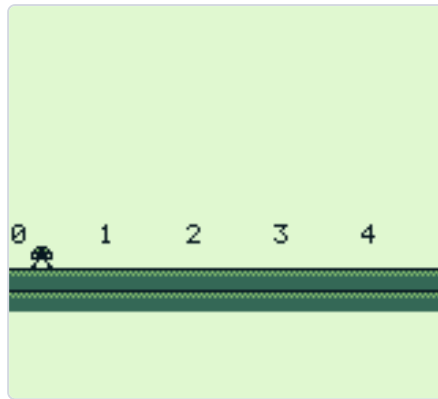
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Le héros part à gauche et marche seul ; au milieu de l'écran, c'est le décor qui défile ; au bout de la carte, il marche de nouveau seul jusqu'au bord.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ : pas de défilement, le héros à 8
- ✓ au milieu du monde, le héros reste au milieu de l'écran (wx 150, cam 74)
- ✓ et le décor défile de cam
- ✓ au bout, la caméra s'arrête à 96
- ✓ et le héros va seul jusqu'au bord : $248 - 96 = 152$ (152)

← [70. Faire glisser le décor](#)

[sommaire](#)

[72. Une fonction à toi : bande\(\)](#) →